Rúbrica del Proyecto Final en Flutter: Etapa 1

Aplicaciones Móviles con Flutter

Duración de la Etapa 1: 2 semanas

Etapa 2: diciembre (Firebase, funcionalidades completas, despliegue)

Modalidad: equipos de 3 integrantes

1. Descripción del proyecto

Cada equipo de 3 estudiantes desarrollará de forma autónoma una aplicación móvil en Flutter, siguiendo la metodología y la arquitectura trabajadas en Vivero+. Debe elegir **una** de las dos rutas:

- **Ruta A: Cliente real.** El equipo busca un cliente (negocio, organización, profesor, emprendedor, asociación) y desarrolla una app adaptada a sus necesidades reales.
- **Ruta B: Problemática social.** El equipo identifica una problemática social y propone una idea de proyecto como solución, justificándola con investigación.

Alcance de la Etapa 1: MVP local

La Etapa 1 entrega un **MVP funcional con datos locales** (lista fija, JSON o assets), **sin Firebase**. La arquitectura debe dejar la app lista para conectar Firebase en la Etapa 2 cambiando solo el Repository, tal como se hizo en Vivero+.

Mínimo técnico exigido en la Etapa 1:

- Flujo de entrada (Splash y/o Onboarding) y navegación entre al menos 4 pantallas.
- Una pantalla de listado/catálogo con búsqueda o filtros.
- Una pantalla de detalle que reciba argumentos.
- Estado compartido con Provider (por ejemplo favoritos, carrito, seguimiento o tareas).
- Tema global y al menos 2 componentes reutilizables propios.
- Imágenes o assets propios declarados en `pubspec.yaml`.
- Capa de datos con Model y Repository.

2. Entregables

#	Entregable	Fecha sugerida
1	Documento de definición (problema o cliente, usuarios, requisitos, alcance del MVP) y bocetos o wireframes	Fin de la semana 1 (primeros 3 días)
2	Repositorio en GitHub con el MVP, README y bitácora de IA (`AI_LOG.md`)	Fin de la semana 2
3	APK o video de la app funcionando	Fin de la semana 2
4	Demo en clase de 5 a 7 minutos	Día de entrega
5	Backlog para la Etapa 2 (incluido en el README)	Fin de la semana 2

3. Rúbrica (100 puntos)

3.1 Criterios comunes (75 puntos)

Criterio	Pts	Excelente	Satisfactorio	En desarrollo	Insuficiente
Definición del problema, usuarios y alcance	10	Problema claro y acotado, usuarios identificados, requisitos priorizados (must / should / could) y alcance realista para 2 semanas	Problema y usuarios claros; requisitos sin priorizar	Problema general o usuarios poco definidos	Sin definición o alcance inviable
Diseño UI/UX y sistema de diseño	15	Tema global (colores, tipografía), 2 o más componentes reutilizables, navegación coherente, buena usabilidad, bocetos previos	Tema y 1 componente reutilizable; pantallas consistentes	Estilos repetidos en cada pantalla; poca coherencia	Sin tema ni consistencia visual
Arquitectura y organización del código	20	Separación por carpetas (core, features, models), Model y Repository, pantallas sin lógica de negocio, nombres claros, código legible	Estructura ordenada; Repository presente pero con algo de lógica mezclada en la UI	Archivos sueltos en `lib/`; lógica mezclada con la interfaz	Todo en un solo archivo
Gestión de estado y navegación	10	Provider bien aplicado (`watch` / `read`), navegación con argumentos, sin errores de estado	Provider funcional con detalles menores	Solo `setState`, o estado compartido con fallas	Sin gestión de estado ni navegación funcional
Funcionalidad del MVP	15	Todos los flujos del mínimo técnico funcionan sin errores; búsqueda o filtros operan; validaciones básicas en formularios	Flujos principales funcionan con fallos menores	Varios flujos incompletos o con errores	La app no compila o no corre
Control de versiones y documentación	5	Commits frecuentes y descriptivos de los 3 integrantes, README con instalación, capturas, decisiones técnicas, backlog de la Etapa 2 y bitácora de IA	Commits regulares; README básico	Pocos commits; README mínimo	Sin repositorio o un solo commit

3.2 Criterios según la ruta elegida (25 puntos)

Ruta A: Cliente real

Criterio	Pts	Excelente	Insuficiente
Evidencia de reuniones con el cliente	8	Al menos 2 reuniones documentadas (minuta, mensajes, fotos) con requisitos confirmados por el cliente	Sin evidencia de contacto con el cliente
Adaptación a las necesidades reales	9	La app responde a lo que el cliente necesita (no a lo que el equipo prefería); decisiones de diseño justificadas con el contexto del cliente	La app es genérica y no refleja al cliente
Validación con el cliente	8	El cliente probó el MVP o un prototipo y hay retroalimentación documentada y un plan para atenderla en la Etapa 2	El cliente no vio ni probó el avance

Ruta B: Problemática social

Criterio	Pts	Excelente	Insuficiente
Investigación de la problemática	8	Problema delimitado con fuentes confiables y datos, población afectada y ODS relacionado	Problema vago, sin fuentes
Originalidad y viabilidad de la solución	9	Idea propia, justificada, factible técnicamente en el tiempo del curso y con impacto potencial claro	Idea copiada, inviable o desconectada del problema
Validación con usuarios potenciales	8	Encuesta, entrevistas o pruebas con al menos 5 personas del público objetivo, con hallazgos documentados	Sin validación

4. Presentación y demo (incluida en los criterios anteriores)

Se evalúa de forma transversal:

- Claridad al explicar el problema y la solución.
- Demo en vivo o video sin interrupciones.
- Cada integrante explica una parte del código y responde preguntas individuales del docente (una pantalla, una decisión de arquitectura o un cambio en vivo).

5. Escala de calificación

Puntos	Nivel
90 a 100	Excelente
80 a 89	Satisfactorio
70 a 79	En desarrollo
Menos de 70	Insuficiente

6. Trabajo en equipo

- **Equipos de 3 integrantes, y todos programan:** nadie puede limitarse a documentar o diseñar.
- **Reparto del trabajo:** cada integrante es responsable principal de al menos una función completa (pantalla, lógica y datos) y apoya al resto. Roles sugeridos y rotativos: coordinación y documentación, diseño UI/UX, arquitectura y Git.
- **Repositorio único:** un solo repositorio, con ramas por función y commits de los 3 integrantes desde su propia cuenta.
- **Defensa individual:** en la demo, cada integrante explica una parte del código y responde preguntas del docente.
- **Factor individual (propuesta):** calificación individual = nota del equipo × factor de defensa. Factor 1.0 si explica con claridad, 0.8 si explica parcialmente y 0.5 si no puede explicar su parte.
- **Problemas en el equipo:** si un integrante no participa, el equipo avisa al docente a más tardar al final de la primera semana.
- **Etapa 2:** no se avanza sin un MVP que compile y corra.
- **Fuentes externas:** el código tomado de tutoriales, repositorios o foros se cita en el README; el código copiado sin citar la fuente se penaliza.

7. Uso limitado de inteligencia artificial

La IA es una herramienta de apoyo, no la autora del proyecto: se permite solo si cada uso queda documentado y justificado. El uso sin documentar no está permitido, y el uso desmedido anula el trabajo.

Usos permitidos (siempre con bitácora)

- Explicar conceptos o mensajes de error.
- Sugerir ideas, alternativas de diseño o de estructura.
- Revisar código propio y señalar posibles errores.
- Generar fragmentos pequeños (referencia: hasta 20 líneas por consulta) que el equipo adapta, entiende y puede explicar.

Usos no permitidos

- Generar pantallas, módulos, funciones completas o la arquitectura del proyecto.
- Redactar el README, la investigación (Ruta B) o la evidencia de reuniones y validación (Ruta A), o inventar datos, minutas y encuestas.
- Pegar código que ningún integrante entiende ni puede explicar.
- Usar IA sin registrarla en la bitácora.